

Exercice 1 :

1. Identifier les états :

Il y a trois états par défaut : Rouge, Orange et Vert.

Il y a également un état d'urgence : Clignotant Orange

2. Identifier les transitions :

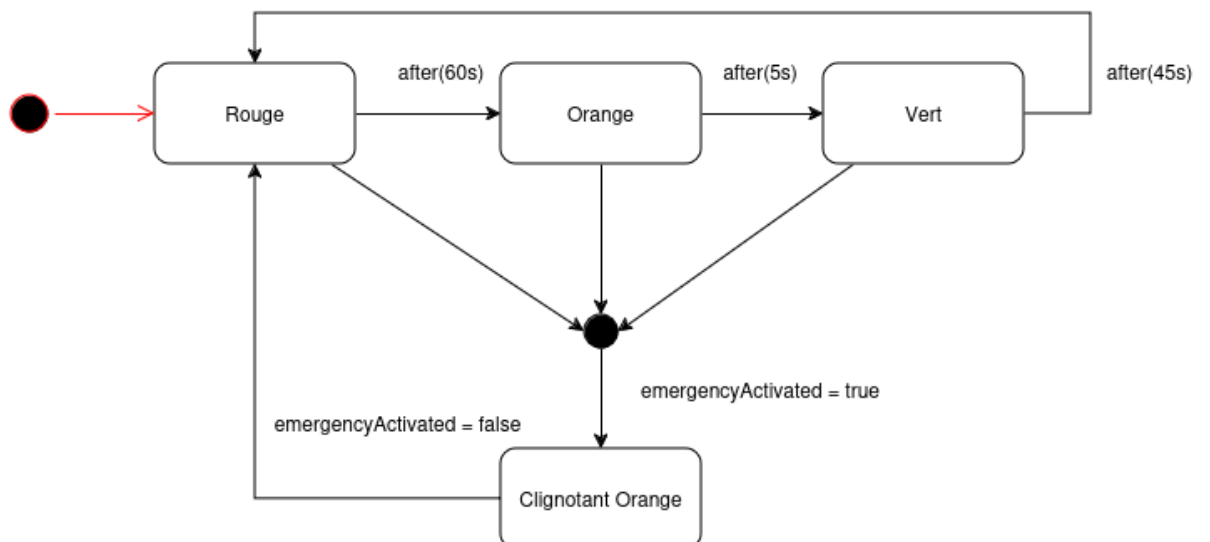
normal:

Rouge 60s -> Orange 5s -> Vert

état d'urgence:

Tout état -> Clignotant Orange désactivation -> Rouge

3. Créer le diagramme UML :



4. Considérez les cas limites :

Dans le cas où le minuteur tomberait en panne, il y a plusieurs cas limites en fonction du type de la panne. Si le minuteur **ne fonctionne plus**, le feu restera dans un état fixe et il n'y aura plus de transition. Dans le cas où le minuteur a une **panne d'horloge** interne il se pourrait qu'il ne respecte pas le temps de transition entre chaque feu. (Par exemple, Rouge 20s -> Vert 1m30s -> Orange 1s -> début)

5. Extension (BONUS)

Exercice 2 :

1. Analyser les états :

Les différents états de la Vending Machine sont :

- Idle
- Waiting for money
- Dispensing Item
- Out of Order

2. Identifier les transitions :

Idle insertMoney() -> DispensingItem

Idle insertMoney() -> WaitingForMoney

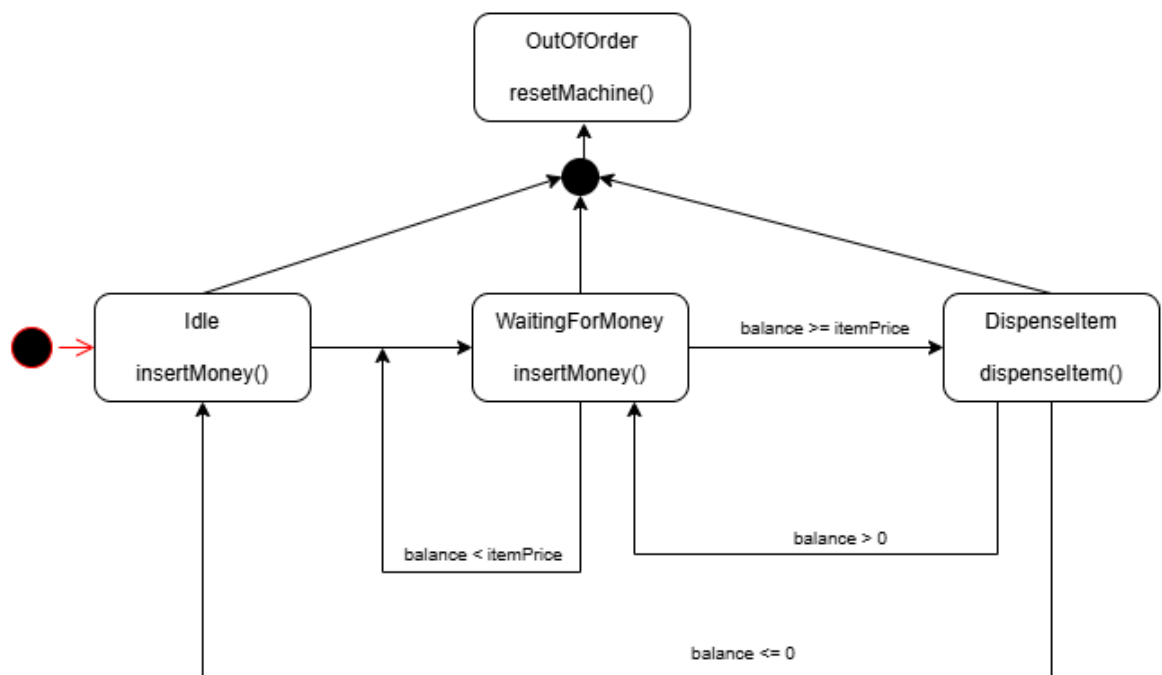
Tout état setOutOfOrder() -> OutOfOrder

OutOfOrder resetMachine() -> Idle

DispensingItem dispenseItem() -> WaitingForMoney

DispensingItem dispenseItem() -> Idle

3. Modéliser la machine à états :



4. Ajouter des gardes fous :

Idle insertMoney() -> (balance >= itemPrice) DispensingItem

Idle insertMoney() -> (balance < itemPrice) WaitingForMoney

Tout état setOutOfOrder() -> OutOfOrder

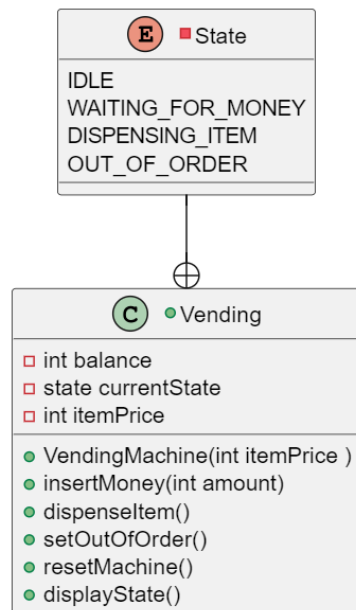
OutOfOrder resetMachine() -> Idle

DispensingItem dispenseItem() -> (balance > 0) WaitingForMoney

DispensingItem dispenseItem() -> (balance <= 0) Idle

5. Utilisez un éditeur UML :

On a envoyé un mail à Monsieur Tibermachine concernant la question 5. Il faut faire la même chose que le 3 mais sur ordinateur (la question 3 devait être effectuée sur feuille). On a quand même laissé le diagramme de classe.



6. Extension (BONUS)